Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №7
на тему
«КРИПТОГРАФИЯ С ИСПОЛЬЗОВАНИЕМ ЭЛЛИПТИЧЕСКИХ КРИВЫХ»
БГУИР 6-05-0612-02 21

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ПОЛЯКОВ Кирилл Борисович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Краткие теоретические сведения 4
2.1 Эллиптическая кривая над простым полем 4
2.2 Задача дискретного логарифмирования 4
2.3 Схема Эль-Гамаля на группе точек 4
2.4 Отображение Коблица 5
3 Выполнение работы 6
3.1 Структура программного средства 6
3.2 Параметры кривых и размер фрагмента 6
3.3 Генерация ключей 7
3.4 Зашифрование 7
3.5 Расшифрование 7
3.6 Интерфейс 8
3.7 Тестирование 10
3.8 Замечания по безопасности 10
Заключение 12
Список использованных источников 13
Приложение А (Обязательное) Листинг программного кода 14

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является изучение криптографии на эллиптических кривых и получение практических навыков построения асимметричных криптосистем над группой точек кривой.

В ходе работы требовалось реализовать программное средство шифрования и дешифрования файлов на основе схемы Эль-Гамаля, перенесённой с мультипликативной группы поля на группу точек эллиптической кривой, в соответствии с методическими указаниями [1]. Для этого необходимо:

1 реализовать генерацию ключевой пары: закрытого ключа d и открытого ключа $Q = dG$;

2 реализовать отображение фрагментов сообщения в точки кривой методом Коблица;

3 реализовать зашифрование, при котором каждому фрагменту соответствует пара точек $(C_1, C_2) = (kG, M + kQ)$;

4 реализовать расшифрование по формуле $M = C_2 - dC_1$ с проверкой корректности точек;

5 переиспользовать арифметику эллиптической кривой и панель асимметричного шифрования, разработанные в предыдущих лабораторных работах;

6 проверить реализацию модульными тестами и встроить её в терминальный интерфейс программы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Эллиптическая кривая над простым полем

Эллиптическая кривая в короткой форме Вейерштрасса над простым полем $\mathrm{GF}(p)$ задаётся уравнением

$$y^2 \equiv x^3 + ax + b \pmod{p}, \quad 4a^3 + 27b^2 \neq 0 \pmod{p}. \tag{2.1}$$

Точки кривой вместе с бесконечно удалённой точкой O образуют абелеву группу. Сложение двух точек $P = (x_1, y_1)$ и $Q = (x_2, y_2)$ при $x_1 \neq x_2$ выполняется по правилу секущей:

$$\lambda = (y_2 - y_1)(x_2 - x_1)^{-1} \bmod p, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1. \tag{2.2}$$

Удвоение точки выполняется по правилу касательной:

$$\lambda = \left(3x_1^2 + a\right)(2y_1)^{-1} \bmod p. \tag{2.3}$$

Сумма точки и ей противоположной $(x, -y)$ равна O; бесконечно удалённая точка служит нейтральным элементом. Умножение точки на число k выполняется двоичным методом «удвоение и сложение» и требует порядка $\log_2 k$ сложений и удвоений [2].

Базовая точка G имеет простой порядок N, то есть $NG = O$. Все вычисления в работе проводятся на двух наборах параметров ГОСТ Р 34.10-2012, уже реализованных в лабораторной работе № 6 [3].

2.2 Задача дискретного логарифмирования

Открытый ключ схемы имеет вид $Q = dG$, где d — закрытый ключ. Противник, знающий Q и G, должен найти d; эта задача называется задачей дискретного логарифмирования в группе точек. Наиболее быстрый известный общий метод решения — алгоритм «шаг младенца, шаг великана» — требует порядка $\sqrt{N}$ операций с точками. Поэтому 256-битный порядок даёт около 2^{128} операций, что и определяет стойкость схемы.

2.3 Схема Эль-Гамаля на группе точек

Схема Эль-Гамаля переносится с мультипликативной группы поля на любую конечную циклическую группу, в том числе на группу точек кривой [4]. Получатель выбирает случайный закрытый ключ d из диапазона $1..N-1$ и публикует $Q = dG$.

Отправитель разбивает сообщение на фрагменты, отображает каждый фрагмент в точку M и для каждого фрагмента выбирает новое случайное k из того же диапазона. Шифртекст фрагмента — пара точек

$$C_1 = kG, \quad C_2 = M + kQ. \tag{2.4}$$

Получатель восстанавливает точку по формуле

$$C_2 - dC_1 = M + kQ - dkG = M + kdG - kdG = M, \tag{2.5}$$

поскольку $Q = dG$. Знание одного C_1 не позволяет найти k: это снова задача дискретного логарифма. Поэтому схема рандомизирована: при повторном зашифровании одного и того же сообщения получается другой шифртекст, и одинаковые фрагменты не видны в данных.

Криптотекст каждого фрагмента занимает две точки, то есть четыре координаты. Это и есть главная плата за асимметричность: расширение данных в несколько раз по сравнению с открытым текстом.

2.4 Отображение Коблица

Чтобы зашифровать произвольные байты, их нужно превратить в точку кривой. Метод Коблица [5] интерпретирует фрагмент как число m и перебирает абсциссы

$$x_j = mK + j, \quad j = 0, 1, \ldots, K - 1. \tag{2.6}$$

Для очередного x_j вычисляется правая часть уравнения кривой $f(x) = x^3 + ax + b$ и проверяется, является ли она квадратичным вычетом по модулю p (критерий Эйлера $f^{\frac{p-1}{2}} \bmod p = 1$). Если да, ордината вычисляется по формуле

$$y = f(x)^{\frac{p+1}{4}} \bmod p, \tag{2.7}$$

которая применима, когда $p \equiv 3 \pmod 4$: показатель целый, а квадрат y равен $f(x)$. Для случайной абсциссы кривая проходит примерно через половину значений, поэтому подходящее j находится в среднем за две попытки, а вероятность того, что все K попыток неудачны, не превосходит 2^{-K}. При $K = 100$ такая неудача практически невозможна.

Обратное преобразование тривиально: $m = x \div K$. Чтобы посторонняя точка не была принята за фрагмент, результат проверяется повторным отображением.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программного средства

Программное средство разделено на три уровня:

– *internal/crypto/ec* — арифметика эллиптической кривой: точки, сложение, удвоение, умножение на число, проверка параметров. Пакет написан в лабораторной работе № 6 и не зависит ни от схемы подписи, ни от схемы шифрования;

– *internal/crypto/ecelgamal* — схема Эль-Гамаля: генерация ключей, отображение Коблица, зашифрование и расшифрование;

– *internal/cipher/elgamal.go* — адаптер к интерфейсу *AsymCipher*, который кодирует ключи и шифртекст в байты;

– *internal/ui/asympane* — готовая панель асимметричного шифрования, переиспользованная без изменений.

Арифметика кривой не дублируется: новый пакет обращается только к готовым функциям *Add*, *Neg* и *ScalarMult*. Это не только сокращает код, но и гарантирует, что обе схемы — подпись ГОСТ Р 34.10-2012 и шифрование Эль-Гамаля — работают на одной и той же проверенной арифметике. Разделение уровней позволяет тестировать математику отдельно от интерфейса.

3.2 Параметры кривых и размер фрагмента

Используются те же два набора параметров, что и в лабораторной работе № 6: модуль $p = 2^{256} - 617$ для 256-битной кривой и $p = 2^{512} - 569$ для 512-битной. Оба модуля сравнимы с 3 по модулю 4, поэтому простая формула квадратного корня $\frac{p+1}{4}$ применима к обеим кривым, и алгоритм Тонелли–Шенкса не нужен. Это проверяется отдельным тестом.

Таблица 3.1 – Параметры кривых и размеры блоков

Кривая	p	$\lvert p\rvert$	$p \bmod 4$	Фрагмент, байт	Блок, байт
256-битная	$2^{256} - 617$	256	3	31	128
512-битная	$2^{512} - 569$	512	3	63	256

Размер фрагмента выбирается по формуле

$$\text{chunkBytes} = (\lvert p\rvert - 8) \div 8, \tag{3.1}$$

где $\lvert p\rvert$ — битовая длина модуля. Восемь бит запаса нужны для множителя K: фрагмент даёт $m < 2^{8 \cdot \text{chunkBytes}} = 2^{\lvert p\rvert - 8}$, поэтому даже при $j = K - 1$

$$mK + j < K \cdot 2^{\lvert p\rvert - 8} = \frac{K}{128} \cdot 2^{\lvert p\rvert - 1} \le p, \tag{3.2}$$

так как у p ровно $|p|$ бит, то есть $p \geq 2^{|p|-1}$, а $K = 100 < 128$. Значит, абсцисса $mK + j$ всегда остаётся в поле и отображение Коблица не может выйти за его границы. Соответствие проверяется тестом *TestChunkBoundKeepsMappingInField* на максимальном фрагменте из одних единичных байт.

3.3 Генерация ключей

Функция *GenerateKey* проверяет параметры кривой, выбирает случайное d из диапазона $1..N - 1$ криптографическим генератором и вычисляет $Q = dG$. Закрытый ключ кодируется числом фиксированной ширины: 32 байта для 256-битной кривой и 64 байта для 512-битной. Открытый ключ кодируется как конкатенация координат $x_Q \parallel y_Q$ той же ширины каждое. Фиксированная ширина важна: по длине ключа адаптер определяет, на какой кривой работать.

3.4 Зашифрование

Сообщение обрамляется служебным заголовком: четыре байта длины сообщения (uint32, старшим байтом вперёд), затем сами байты. Обрамлённая строка дополняется нулями до целого числа фрагментов. Заголовок входит в полезную нагрузку, поэтому даже пустое сообщение даёт один блок; иначе расшифрование не смогло бы отличить пустое сообщение от отсутствия данных.

Каждый фрагмент отображается в точку M методом Коблица. Затем выбирается случайное k и вычисляются $C_1 = kG$ и $C_2 = M + kQ$. В шифртекст дописываются четыре координаты C_1 и C_2 по $|p|$ байт каждая, то есть блок занимает $4\,|p|$ байт. Схема зашифрования показана на рисунке 3.1.

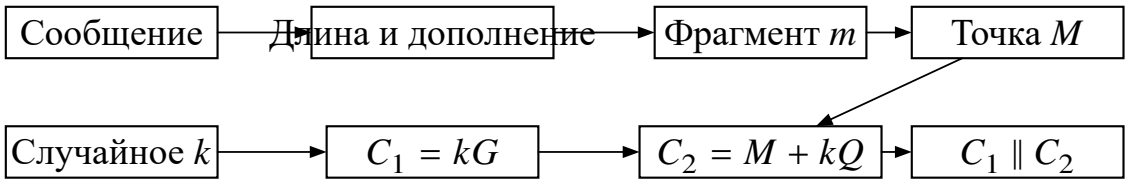


Рисунок 3.1 – Схема зашифрования Эль-Гамаля на эллиптической кривой

3.5 Расшифрование

Для каждого блока проверяется, что точки C_1 и C_2 лежат на кривой и не являются бесконечно удалёнными, после чего вычисляется

$$M = C_2 - dC_1. \tag{3.3}$$

Из абсциссы восстанавливается фрагмент $m = x_M \div K$; точка принимается только в том случае, если повторное отображение Коблица даёт ту же самую точку. Это отсекает посторонние точки, которые возникают при неверном ключе или повреждённом шифртексте. Полученные фрагменты склеиваются,

из заголовка берётся длина сообщения, и нулевое дополнение отбрасывается. Схема расшифрования показана на рисунке 3.2.

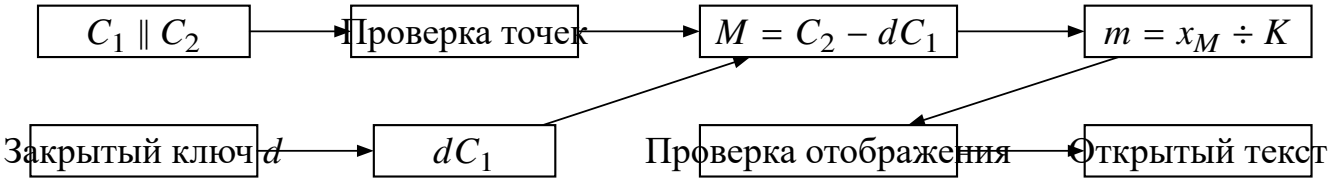


Рисунок 3.2 – Схема расшифрования Эль-Гамаля на эллиптической кривой

Ошибками, а не посторонним результатом, являются: пустой шифртекст, длина, не кратная блоку, точка не на кривой, бесконечно удалённая точка, точка, не являющаяся отображением фрагмента, а также объявленная длина сообщения, которая больше полезной нагрузки. Все эти случаи проверены отрицательными тестами.

3.6 Интерфейс

Собственная панель для лабораторной работы не создавалась. Схема Эль-Гамаля полностью укладывается в уже существующую панель *asympane*: та же генерация ключевой пары, тот же выбор размера параметров (256 или 512 бит), тот же режим *Encrypt* с открытым ключом и режим *Decrypt* с закрытым. Поэтому в *main.go* добавлена только новая вкладка *EC ElGamal* сам пакет панели не изменён.

Клавиша *Ctrl+G* генерирует ключевую пару, *Ctrl+R* показывает закрытый ключ, *Ctrl+K* очищает выбранное поле. Входные данные можно ввести вручную или выбрать файл, шифртекст вводится и выводится в шестнадцатеричном виде, результат сохраняется в файл кнопкой *SAVE TO FILE*. Режим переключается по *Enter* на поле *MODE*, запуск — по *Enter* на кнопке *RUN*. *Ctrl+D* переносит шифртекст из вывода во ввод, чтобы сразу расшифровать его тем же ключом.



Рисунок 3.3 – Вкладка *EC ElGamal*: ключевая пара и шифртекст из двух блоков для сообщения в 16 байт

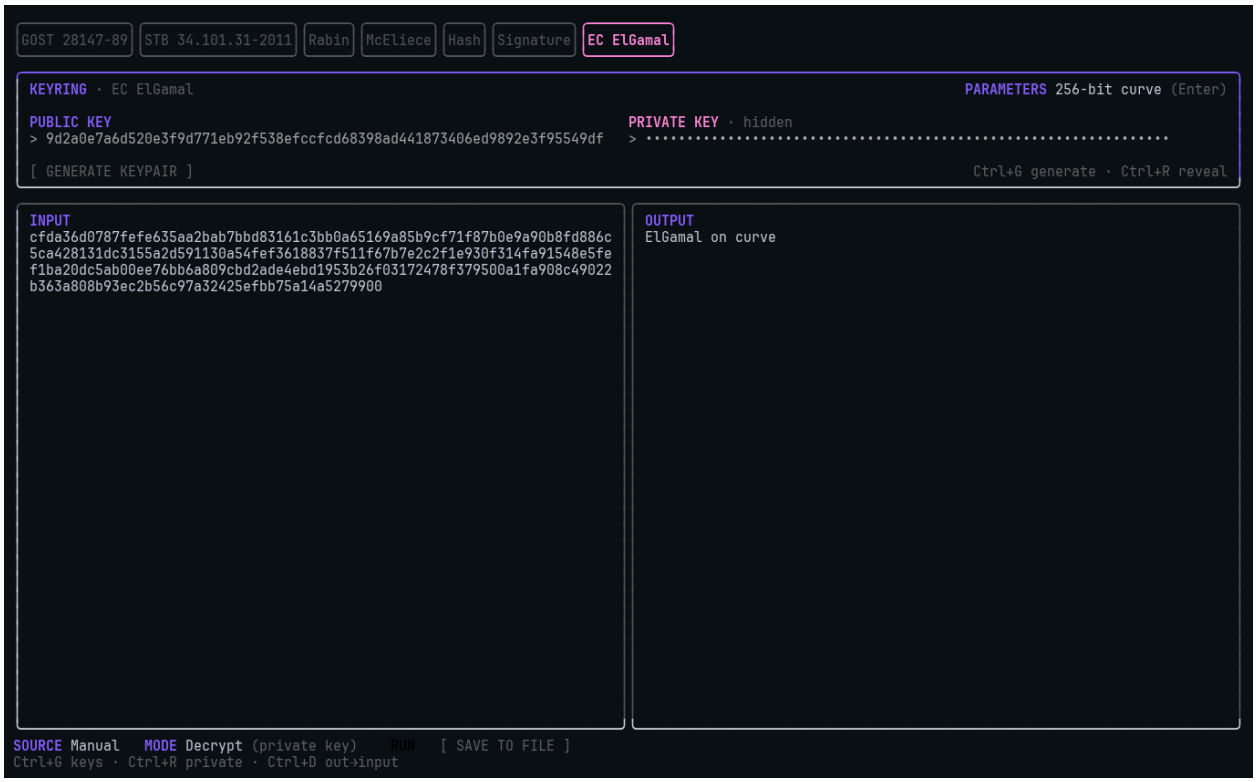


Рисунок 3.4 – Тот же экран в режиме *Decrypt*: сообщение восстановлено

3.7 Тестирование

Корректность проверена командой `go test ./...`. Все тесты проходят.

– *TestParamSetsSupportShortSquareRoot* проверяет, что оба модуля сравнимы с 3 по модулю 4 и что размеры фрагментов равны 31 и 63 байтам.

– *TestChunkBoundKeepsMappingInField* проверяет границу $mK + j < p$ на максимальном фрагменте.

– *TestMessageMappingCoversEveryByte* выполняет отображение и обратное преобразование для всех 256 значений байта на обеих кривых; проверяется, что точка лежит на кривой, а абсцисса имеет вид $mK + j$.

– *TestRoundTrip* выполняет цикл «зашифрование–расшифрование» для пустого сообщения, одного байта, короткого текста, сообщения из нулевых байтов, случайных двоичных данных и сообщений ровно на границе фрагмента.

– *TestCiphertextExpansion* проверяет, что длина шифртекста равна $4\,|p|$ байт на блок.

– *TestDeterministicNonce* подтверждает, что при одинаковом k шифртекст совпадает, а при другом k — различается.

– *TestWrongKeyDoesNotRecoverMessage* проверяет, что чужой закрытый ключ не возвращает исходное сообщение.

– *TestDecryptRejectsMalformed* и *TestInvalidKeysRejected* проверяют отказ при повреждённом шифртексте, точках не с кривой, бесконечно удалённых точках и ключах вне диапазона.

Измеренное расширение данных на реальных запусках приведено в таблице 3.2. Для 256-битной кривой один фрагмент полезной нагрузки — это 31 байт (с учётом заголовка), для 512-битной — 63 байта; каждому фрагменту соответствует блок из 128 или 256 байт.

Таблица 3.2 – Измеренное расширение сообщения

Сообщение	256-битная кривая	512-битная кривая
15 байт	1 блок, 128 Б (× 8.53)	1 блок, 256 Б (× 17.07)
96 байт	4 блока, 512 Б (× 5.33)	2 блока, 512 Б (× 5.33)
1000 байт	33 блока, 4224 Б (× 4.22)	16 блоков, 4096 Б (× 4.10)

В терминальном интерфейсе на 256-битной кривой фраза «Привет, Эль-Гамаль на кривой!» длиной 52 байта зашифровалась в 512 шестнадцатеричных символов, то есть в 256 байт (два блока), и после переключения в режим расшифрования восстановилась без потерь.

3.8 Замечания по безопасности

– Одноразовый множитель k берётся из *crypto/rand* заново для каждого фрагмента. Повторное использование k при одном ключе раскрывает открытый

текст: из пары $C_2 - C_{2'} = M - M'$ при известном одном из сообщений восстанавливается второе.

– Перед расшифрованием все точки проверяются на принадлежность кривой и отсутствие бесконечно удалённой точки; результат отображения проверяется повторным отображением.

– Реализация учебная: *math/big* не обеспечивает постоянного времени вычислений, поэтому она не защищена от атак по времени и побочным каналам.

– Схема не добавляет к сообщению случайной избыточности сверх рандомизации k; для практических систем применяется гибридная схема (Эль-Гамаль для ключа, симметричный шифр для данных), которая к тому же снимает проблему расширения.

ЗАКЛЮЧЕНИЕ

В ходе работы реализована схема шифрования Эль-Гамаля на группе точек эллиптической кривой. Арифметика кривой переиспользована из лабораторной работы № 6; поверх неё построены генерация ключевой пары, отображение фрагментов сообщения в точки методом Коблица, зашифрование парой точек $(C_1, C_2) = (kG, M + kQ)$ и расшифрование по формуле $M = C_2 - dC_1$.

Показано, что оба набора параметров ГОСТ Р 34.10-2012 допускают простую формулу квадратного корня $\frac{p+1}{4}$, поскольку их модули сравнимы с 3 по модулю 4, а выбранный размер фрагмента $(|p| - 8) \div 8$ гарантирует $mK + j < p$ для любого фрагмента. Схема встроена в существующую панель асимметричного шифрования, поэтому отдельный интерфейс не потребовался.

Корректность подтверждена тестами на обеих кривых, отображением всех 256 значений байта и отрицательными тестами на повреждённый шифртекст и чужие ключи. Измеренное расширение составляет около 4,2 раза для длинных сообщений и до 17 раз для коротких, что объясняется затратой двух точек на каждый фрагмент.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Лабораторные работы по дисциплине «Методы защиты информации» : методические указания. – 2024.

[2] Menezes, A.J. Handbook of Applied Cryptography / A.J. Menezes, P.C. van Oorschot, S.A. Vanstone. – Boca Raton: CRC Press, 1996. – 780 с.

[3] Smyshlyaev, S. Cryptographic Algorithms for the GOST R 34.10-2012 and GOST R 34.11-2012 Standards / S. Smyshlyaev [и др.]. – 2016.

[4] ElGamal, T. A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms / T. ElGamal // IEEE Transactions on Information Theory. – 1985. – Т. 31, № 4. – С. 469-472.

[5] Koblitz, N. Elliptic Curve Cryptosystems / N. Koblitz // Mathematics of Computation. – 1987. – Т. 48, № 177. – С. 203-209.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Схема Эль-Гамаля на эллиптической кривой

```
// Пакет ecelgamal реализует схему шифрования Эль-Гамаля на группе точек
// эллиптической кривой над простым полем.
//
// Обозначения: G — базовая точка порядка N, d — закрытый ключ, Q = d*G —
// открытый ключ, k — одноразовый случайный множитель. Шифртекст каждого
// блока — пара точек (C1, C2) = (k*G, M + k*Q), где M — точка, в которую
// отображён очередной фрагмент сообщения методом Коблица.
//
// Арифметика кривой берётся из пакета ec; здесь нет ни одной операции над
// точками, кроме композиции готовых функций Add, Neg и ScalarMult.
package ecelgamal

import (
  "crypto/rand"
  "encoding/binary"
  "errors"
  "fmt"
  "io"
  "math/big"

  "go-cipher/internal/crypto/ec"
)

const (
  // K — параметр отображения Коблица: байтовому блоку m ищется точка с
  // абсциссой x = m*K + j, j = 0..K-1. Для случайного x кривая проходит
  // примерно через половину значений абсцисс, поэтому подходящее j
  // находится за ~2 попытки, а вероятность неудачи после K попыток не
  // превосходит 2^-K.
  K = 100

  // headerBytes — длина поля с размером сообщения в начале полезной
  // нагрузки. Размер записывается как uint32, старшим байтом вперёд.
  headerBytes = 4
)

var (
  // ErrPublicKey — открытый ключ не является точкой кривой порядка N.
  ErrPublicKey = errors.New("ecelgamal: invalid public key")
  // ErrPrivateKey — закрытый ключ не лежит в диапазоне 1..N-1.
  ErrPrivateKey = errors.New("ecelgamal: private key out of range")
  // ErrCiphertext — шифртекст неверной длины, содержит точку не с
  // кривой или бесконечно удалённую точку.
  ErrCiphertext = errors.New("ecelgamal: malformed ciphertext")
  // ErrChunk — фрагмент сообщения длиннее, чем допускает размер поля.
  ErrChunk = errors.New("ecelgamal: message chunk is too long")
  // ErrMapping — для фрагмента не нашлось точки кривой (отображение
  // Коблица не удалось) либо точка не соответствует никакому фрагменту.
  ErrMapping = errors.New("ecelgamal: no curve point maps the chunk")
  // ErrUnsupportedPrime — модуль кривой не сравним с 3 по модулю 4,
  // поэтому простой формулы квадратного корня (p+1)/4 недостаточно.
  ErrUnsupportedPrime = errors.New("ecelgamal: curve prime is not 3 mod 4")
  // ErrNonce — не удалось получить случайное k.
  ErrNonce = errors.New("ecelgamal: cannot generate nonce")
)
```

```
var (
  one = big.NewInt(1)
  // kInt — параметр K в виде big.Int, чтобы не создавать его на каждом
  // шаге отображения.
  kInt = big.NewInt(K)
)

// GenerateKey формирует ключевую пару: случайный закрытый ключ d из
// диапазона 1..N-1 и открытый ключ Q = d*G.
func GenerateKey(c *ec.Curve, rnd io.Reader) (*big.Int, ec.Point, error) {
  if err := validateCurve(c); err != nil {
    return nil, ec.Point{}, err
  }

  d, err := randomScalar(c.N, rnd)
  if err != nil {
    return nil, ec.Point{}, err
  }
  return d, c.ScalarMult(d, c.Base()), nil
}

// Encrypt зашифровывает сообщение msg открытым ключом public.
//
// Сообщение обрамляется заголовком: uint32 с длиной сообщения в байтах,
// затем сами байты. Обрамлённая строка дополняется нулями до целого числа
// фрагментов по ChunkBytes(c) байт. Каждый фрагмент отображается в точку
// M, после чего для случайного k из 1..N-1 вычисляются
//
//  C1 = k*G,  C2 = M + k*public,
//
// и в шифртекст дописываются четыре координаты C1 и C2 фиксированной
// ширины, по |p| байт каждая.
func Encrypt(c *ec.Curve, public ec.Point, msg []byte, rnd io.Reader) ([]byte,
error) {
  if err := validateCurve(c); err != nil {
    return nil, err
  }
  if err := validatePublicKey(c, public); err != nil {
    return nil, err
  }
  if rnd == nil {
    return nil, fmt.Errorf("%w: nil reader", ErrNonce)
  }
  if uint64(len(msg)) > 1<<32-1 {
    return nil, fmt.Errorf("%w: message is %d bytes", ErrChunk, len(msg))
  }

  width := coordinateWidth(c)
  chunkBytes := ChunkBytes(c)

  // Обрамление: длина сообщения, затем его байты, затем нулевое
  // дополнение до целого числа фрагментов. Заголовок входит в полезную
  // нагрузку, поэтому блоков всегда хотя бы один даже для пустого
  // сообщения — иначе расшифрование не смогло бы отличить пустое
  // сообщение от отсутствия данных.
  payload := make([]byte, headerBytes+len(msg))
  binary.BigEndian.PutUint32(payload[:headerBytes], uint32(len(msg)))
  copy(payload[headerBytes:], msg)

  blocks := (len(payload) + chunkBytes - 1) / chunkBytes
  cipherText := make([]byte, 0, blocks*4*width)

  for i := 0; i < blocks; i++ {
```

```
    chunk := make([]byte, chunkBytes)
    copy(chunk, payload[i*chunkBytes:])

    // M — точка, изображающая фрагмент.
    m, err := MessageToPoint(c, chunk)
    if err != nil {
      return nil, err
    }

    // k — одноразовый множитель; C1 = k*G, C2 = M + k*Q.
    k, err := randomScalar(c.N, rnd)
    if err != nil {
      return nil, err
    }
    c1 := c.ScalarMult(k, c.Base())
    c2 := c.Add(m, c.ScalarMult(k, public))

    cipherText = appendPoint(cipherText, c1, width)
    cipherText = appendPoint(cipherText, c2, width)
  }

  return cipherText, nil
}

// Decrypt расшифровывает шифртекст ct закрытым ключом priv.
//
// Для каждого блока восстанавливается точка
//
//  M = C2 - d*C1 = M + k*Q - d*k*G = M,
//
// поскольку Q = d*G. Из абсциссы M извлекается номер фрагмента m = x/K,
// после чего отбрасывается нулевое дополнение по записанной в заголовке
// длине сообщения.
func Decrypt(c *ec.Curve, priv *big.Int, cipherText []byte) ([]byte, error) {
  if err := validateCurve(c); err != nil {
    return nil, err
  }
  if priv == nil || priv.Sign() <= 0 || priv.Cmp(c.N) >= 0 {
    return nil, ErrPrivateKey
  }

  width := coordinateWidth(c)
  blockBytes := 4 * width
  if len(cipherText) == 0 || len(cipherText)%blockBytes != 0 {
   return nil, fmt.Errorf("%w: length %d is not a multiple of %d", ErrCiphertext,
len(cipherText), blockBytes)
  }

  chunkBytes := ChunkBytes(c)
  blocks := len(cipherText) / blockBytes
  payload := make([]byte, 0, blocks*chunkBytes)

  for i := 0; i < blocks; i++ {
    block := cipherText[i*blockBytes : (i+1)*blockBytes]
    c1, c2 := parseBlock(block, width)
   if c1.IsInfinity() || c2.IsInfinity() || !c.IsOnCurve(c1) || !c.IsOnCurve(c2)
{
       return nil, fmt.Errorf("%w: block %d is not a pair of curve points",
ErrCiphertext, i)
    }

    // M = C2 - d*C1.
    m := c.Add(c2, c.Neg(c.ScalarMult(priv, c1)))
```

```
    if m.IsInfinity() {
      return nil, fmt.Errorf("%w: block %d decrypts to the point at infinity",
ErrCiphertext, i)
    }

    chunk, err := PointToMessage(c, m, chunkBytes)
    if err != nil {
      return nil, fmt.Errorf("%w: block %d", err, i)
    }
    payload = append(payload, chunk...)
  }

  if len(payload) < headerBytes {
       return nil, fmt.Errorf("%w: payload is shorter than the header",
ErrCiphertext)
  }
  length := binary.BigEndian.Uint32(payload[:headerBytes])
  if uint64(length) > uint64(len(payload)-headerBytes) {
       return nil, fmt.Errorf("%w: declared length %d exceeds %d bytes",
ErrCiphertext, length, len(payload)-headerBytes)
  }

  return payload[headerBytes : headerBytes+int(length)], nil
}

// MessageToPoint отображает фрагмент сообщения в точку кривой методом
// Коблица. Байты фрагмента интерпретируются как число m; перебираются
// абсциссы x = m*K + j, j = 0..K-1, и берётся первая, для которой
// правая часть уравнения кривой
//
//  f(x) = x^3 + a*x + b (mod p)
//
// является квадратичным вычетом. Ордината вычисляется по формуле
//
//  y = f(x)^((p+1)/4) mod p,
//
// которая верна для простого p = 3 (mod 4) и даёт один из двух корней.
func MessageToPoint(c *ec.Curve, chunk []byte) (ec.Point, error) {
  if err := validateCurve(c); err != nil {
    return ec.Point{}, err
  }
  chunkBytes := ChunkBytes(c)
  if len(chunk) > chunkBytes {
       return ec.Point{}, fmt.Errorf("%w: %d bytes, at most %d", ErrChunk,
len(chunk), chunkBytes)
  }
  if !primeIs3Mod4(c.P) {
     return ec.Point{}, fmt.Errorf("%w: p mod 4 = %d", ErrUnsupportedPrime,
new(big.Int).Mod(c.P, big.NewInt(4)))
  }

  m := new(big.Int).SetBytes(chunk)
  for j := 0; j < K; j++ {
    // x = m*K + j; при выбранном размере фрагмента x < p гарантировано.
    x := new(big.Int).Mul(m, kInt)
    x.Add(x, big.NewInt(int64(j)))
    if x.Cmp(c.P) >= 0 {
      break
    }

    // f(x) = x^3 + a*x + b (mod p).
    fx := curveRightHandSide(c, x)
    y, ok := squareRoot(c.P, fx)
```

```
    if !ok {
      continue
    }

    p := ec.Point{X: x, Y: y}
    // Уравнению удовлетворяет и точка с ординатой p - y; берётся
    // корень, который возвращает squareRoot, чтобы отображение было
    // однозначным и обратное преобразование могло проверить результат
    // повторным отображением.
    if !c.IsOnCurve(p) {
      continue
    }
    return p, nil
  }

  return ec.Point{}, fmt.Errorf("%w: no j in 0..%d", ErrMapping, K-1)
}

// PointToMessage восстанавливает фрагмент сообщения из точки M: абсцисса
// имеет вид x = m*K + j, поэтому m = x/K. Результат проверяется повторным
// отображением: принимается только та точка, которую MessageToPoint
// порождает для полученного фрагмента. Это отвергает постороннюю точку,
// возникшую при неверном ключе или повреждённом шифртексте.
func PointToMessage(c *ec.Curve, p ec.Point, chunkBytes int) ([]byte, error) {
  if err := validateCurve(c); err != nil {
    return nil, err
  }
  if p.IsInfinity() {
    return nil, fmt.Errorf("%w: point at infinity", ErrCiphertext)
  }
  if !c.IsOnCurve(p) {
    return nil, fmt.Errorf("%w: %v", ErrCiphertext, ec.ErrPointNotOnCurve)
  }

  // m = x / K, остаток j = x mod K должен лежать в 0..K-1, что верно по
  // определению деления с остатком.
  m := new(big.Int).Quo(p.X, kInt)
  if m.BitLen() > 8*chunkBytes {
     return nil, fmt.Errorf("%w: x coordinate %s does not encode %d bytes",
ErrMapping, p.X, chunkBytes)
  }
  chunk := m.FillBytes(make([]byte, chunkBytes))

  check, err := MessageToPoint(c, chunk)
  if err != nil {
    return nil, err
  }
  if !check.Equal(p) {
     return nil, fmt.Errorf("%w: x coordinate %s is not a message mapping",
ErrMapping, p.X)
  }
  return chunk, nil
}

// ChunkBytes возвращает длину фрагмента сообщения в байтах:
//
//  chunkBytes = (|p| - 8) / 8,
//
// где |p| — битовая длина модуля. Запас в восемь бит нужен для множителя K:
// m < 2^(8*chunkBytes) = 2^(|p|-8), поэтому
//
//  m*K + j < K*2^(|p|-8) = (K/128)*2^(|p|-1) <= p,
//
```

```
// так как у p ровно |p| бит, то есть p >= 2^(|p|-1), а K = 100 < 128.
func ChunkBytes(c *ec.Curve) int {
  if c == nil || c.P == nil {
    return 0
  }
  return (c.P.BitLen() - 8) / 8
}

// validateCurve проверяет параметры кривой и пригодность простой формулы
// квадратного корня.
func validateCurve(c *ec.Curve) error {
  if err := c.Validate(); err != nil {
    return err
  }
  if !primeIs3Mod4(c.P) {
          return  fmt.Errorf("%w:  p  mod  4  =  %d",  ErrUnsupportedPrime,
new(big.Int).Mod(c.P, big.NewInt(4)))
  }
  return nil
}

// validatePublicKey проверяет, что открытый ключ — точка кривой порядка N.
func validatePublicKey(c *ec.Curve, public ec.Point) error {
  if public.IsInfinity() || !c.IsOnCurve(public) {
    return ErrPublicKey
  }
  if !c.ScalarMult(c.N, public).IsInfinity() {
    return fmt.Errorf("%w: point order is not N", ErrPublicKey)
  }
  return nil
}

// primeIs3Mod4 сообщает, сравним ли модуль с 3 по модулю 4.
func primeIs3Mod4(p *big.Int) bool {
  return new(big.Int).And(p, big.NewInt(3)).Cmp(big.NewInt(3)) == 0
}

// curveRightHandSide вычисляет f(x) = x^3 + a*x + b (mod p).
func curveRightHandSide(c *ec.Curve, x *big.Int) *big.Int {
  fx := new(big.Int).Mul(x, x)
  fx.Mul(fx, x)
  fx.Add(fx, new(big.Int).Mul(c.A, x))
  fx.Add(fx, c.B)
  fx.Mod(fx, c.P)
  return fx
}

// squareRoot возвращает квадратный корень из f по модулю p = 3 (mod 4) по
// формуле f^((p+1)/4). ok = false, если f не является квадратичным вычетом.
// Отдельно обрабатывается f = 0: корень равен нулю, это точка порядка 2.
func squareRoot(p, f *big.Int) (*big.Int, bool) {
  if f.Sign() == 0 {
    return big.NewInt(0), true
  }

  // Критерий Эйлера: f^((p-1)/2) = 1 для ненулевого квадратичного
  // вычета и -1 иначе.
   legendre  := new(big.Int).Exp(f,  new(big.Int).Rsh(new(big.Int).Sub(p,  one),
1), p)
  if legendre.Cmp(one) != 0 {
    return nil, false
  }
```

```
  // f^((p+1)/4): при p = 3 (mod 4) показатель целый, а квадрат корня
  // равен f.
  exponent := new(big.Int).Add(p, one)
  exponent.Rsh(exponent, 2)
  y := new(big.Int).Exp(f, exponent, p)

  check := new(big.Int).Mul(y, y)
  check.Mod(check, p)
  if check.Cmp(f) != 0 {
    return nil, false
  }
  return y, true
}

// randomScalar выбирает k из диапазона 1..n-1, читая байты из rnd.
func randomScalar(n *big.Int, rnd io.Reader) (*big.Int, error) {
  for {
    k, err := rand.Int(rnd, new(big.Int).Sub(n, one))
    if err != nil {
      return nil, fmt.Errorf("%w: %v", ErrNonce, err)
    }
    if k.Sign() != 0 {
      return k, nil
    }
  }
}

// coordinateWidth возвращает ширину координаты в байтах, |p|/8.
func coordinateWidth(c *ec.Curve) int {
  return (c.P.BitLen() + 7) / 8
}

// appendPoint дописывает к buf координаты точки x||y фиксированной ширины.
func appendPoint(buf []byte, p ec.Point, width int) []byte {
  start := len(buf)
  buf = append(buf, make([]byte, 2*width)...)
  p.X.FillBytes(buf[start : start+width])
  p.Y.FillBytes(buf[start+width : start+2*width])
  return buf
}

// parseBlock разбирает блок C1||C2 на две точки.
func parseBlock(block []byte, width int) (ec.Point, ec.Point) {
  c1 := ec.Point{
    X: new(big.Int).SetBytes(block[:width]),
    Y: new(big.Int).SetBytes(block[width : 2*width]),
  }
  c2 := ec.Point{
    X: new(big.Int).SetBytes(block[2*width : 3*width]),
    Y: new(big.Int).SetBytes(block[3*width:]),
  }
  return c1, c2
}
```

Листинг А.2 – Адаптер к интерфейсу программы

```
package cipher

import (
  "crypto/rand"
  "math/big"

  "go-cipher/internal/crypto/ec"
```

```
  "go-cipher/internal/crypto/ecelgamal"
)

// ElGamal — адаптер схемы Эль-Гамаля на эллиптических кривых к интерфейсу
// AsymCipher.
//
// Ключи кодируются числами фиксированной ширины, равной длине модуля p в
// байтах: 32 байта для 256-битной кривой и 64 для 512-битной. Открытый ключ
// — x||y точки Q, закрытый — само число d. Шифртекст — последовательность
// блоков C1||C2, которые формирует пакет ecelgamal.
type ElGamal struct{}

func NewElGamal() ElGamal {
  return ElGamal{}
}

func (ElGamal) Name() string {
  return "EC ElGamal"
}

func (ElGamal) KeyOptions() []KeyOption {
  return []KeyOption{
    {Label: "256-bit curve", Bits: 256},
    {Label: "512-bit curve", Bits: 512},
  }
}

func (ElGamal) GenerateKeys(option KeyOption) (KeyPair, error) {
  c, err := curveByBits(option.Bits)
  if err != nil {
    return KeyPair{}, err
  }

  priv, pub, err := ecelgamal.GenerateKey(c, rand.Reader)
  if err != nil {
    return KeyPair{}, err
  }

  width := numberWidth(c)
  // Открытый ключ — x||y, закрытый — d.
  public := make([]byte, 2*width)
  pub.X.FillBytes(public[:width])
  pub.Y.FillBytes(public[width:])

  private := make([]byte, width)
  priv.FillBytes(private)

  return KeyPair{Public: public, Private: private}, nil
}

func (ElGamal) Encrypt(pub, msg []byte) ([]byte, error) {
  c, err := curveByPublic(pub)
  if err != nil {
    return nil, err
  }

  width := numberWidth(c)
  q := ec.Point{
    X: new(big.Int).SetBytes(pub[:width]),
    Y: new(big.Int).SetBytes(pub[width:]),
  }

  return ecelgamal.Encrypt(c, q, msg, rand.Reader)
```

```
}

func (ElGamal) Decrypt(priv, cipherText []byte) ([]byte, error) {
  c, err := curveByPrivate(priv)
  if err != nil {
    return nil, err
  }

  d := new(big.Int).SetBytes(priv)
  return ecelgamal.Decrypt(c, d, cipherText)
}
```